

ServiceMate-Local Service Booking Platform

1. System Overview

ServiceMate is a web-based service booking platform that connects customers with local service providers. Users can browse available services, book providers, and track booking status through a simple interface.

The system follows a three-tier architecture consisting of the Frontend (Presentation Layer), Backend (Application Layer), and Database (Data Layer).

2. High-Level Architecture Components

1. Client / User Interface (Frontend)

- Built using React.js
- Provides user interaction
- Allows users to:
 - Register/Login
 - Browse services
 - Book service providers
 - View booking history

2. Backend / Application Server

- Built using Spring Boot
- Handles business logic
- Processes API requests
- Manages authentication and booking operations
- **Responsibilities:**
 - User authentication
 - Service management
 - Booking management
 - Communication with database

3. Database Layer

- Uses MySQL
- Stores all system data
- **Main data stored:**
 - User information
 - Service provider details
 - Service categories
 - Booking records
 - Booking status

3. Core System Components

- **User Module**
 - Handles: User registration, Login authentication, Profile management
- **Service Module**
 - Handles: Listing service categories, Viewing available providers
- **Booking Module**
 - Handles: Creating service requests, Updating booking status, Tracking service progress
- **Provider Module**
 - Allows service providers to: View incoming requests, Accept or reject bookings, Update service completion

4. Data Flow

1. User accesses the React frontend
2. User sends request (login / booking / service search)
3. Request goes to Spring Boot REST API
4. Backend processes request
5. Backend interacts with MySQL database
6. Response is returned to frontend
7. Frontend displays updated information to the user

5. Technology Stack

Layer	Technology
Frontend	React.js
Backend	Spring Boot
Database	MySQL
Version Control	GitHub
API Communication	REST APIs

6. Non-Functional Requirements

- **Scalability** – System should support increasing users.
 - **Security** – Password encryption and authentication.
 - **Availability** – System accessible anytime.
 - **Performance** – Fast response for service bookings.
- Low Level System Design (LLD)

ServiceMate – Low Level Design

1. Module Level Design

The ServiceMate system is divided into the following modules:

1. User Module

Responsible for user account management.

- **Functions:**

- User registration
- Login authentication
- Profile management
- Viewing booking history

- **Main APIs:**

- POST /register
- POST /login
- GET /profile

2. Service Module

Handles available service categories and provider listings.

- **Functions:**

- Display available services
- Show providers for selected service
- Manage service details

- **Main APIs:**

- GET /services
- GET /services/{id}

3. Booking Module

Handles service booking and tracking.

- **Functions:**

- Create booking request
- Assign provider
- Track service status
- Update booking completion
-

- **Main APIs:**

- POST /book-service

- GET /user-bookings
- PUT /update-booking-status

4. Provider Module

Allows service providers to manage incoming service requests.

- **Functions:**
 - View booking requests
 - Accept or reject bookings
 - Update service progress
- **Main APIs:**
 - GET /provider/bookings
 - PUT /provider/accept-booking
 - PUT /provider/reject-booking

2. Database Table Design

Table Name	Field	Type	Description
Users	user_id ▾	INT (PK) ▾	Unique user ID ▾
	name ▾	VARCHAR ▾	User name ▾
	email ▾	VARCHAR ▾	Email address ▾
	password ▾	VARCHAR ▾	Encrypted pass... ▾
	phone ▾	VARCHAR ▾	Contact number ▾
Service_Providers	provider_id ▾	INT (PK) ▾	Unique provide... ▾
	name ▾	VARCHAR ▾	Provider name ▾
	service_type ▾	VARCHAR ▾	Type of service ▾
	phone ▾	VARCHAR ▾	Contact number ▾
	location ▾	VARCHAR ▾	Service location ▾
Services	service_id ▾	INT (PK) ▾	Service ID ▾

	service_name ▾	VARCHAR ▾	Service name ▾
	description ▾	TEXT ▾	Service descrip... ▾
Bookings	booking_id ▾	INT (PK) ▾	Booking ID ▾
	user_id ▾	INT (FK) ▾	User reference ▾
	provider_id ▾	INT (FK) ▾	Provider refere... ▾
	service_id ▾	INT (FK) ▾	Service reference ▾
	booking_date ▾	DATE ▾	Date of booking ▾
	status ▾	VARCHAR ▾	Pending / Acce... ▾

3. Class Design

Main Classes

- **User**
 - **Attributes:** userId, name, email, password, phone
 - **Methods:** register(), login(), viewBookings()
- **Service**
 - **Attributes:** serviceId, serviceName, description
 - **Methods:** getServices(), getServiceDetails()
- **ServiceProvider**
 - **Attributes:** providerId, name, serviceType, phone, location
 - **Methods:** viewRequests(), acceptBooking(), rejectBooking()
- **Booking**
 - **Attributes:** bookingId, userId, providerId, serviceId, bookingDate, status
 - **Methods:** createBooking(), updateStatus(), viewBooking()

4. API Request Flow

Example: Service Booking

1. User selects a service from frontend.

Frontend sends request to backend API:

POST /book-service

Request Body:

2. { "user_id": 101, "service_id": 3, "provider_id": 22 }
3. Backend validates request.
4. Booking record stored in MySQL database.
5. Provider receives booking request.

5. Error Handling

System handles common errors such as:

- Invalid login credentials
- Service not available
- Booking request failure
- Database connection errors

Error responses are returned through the API.

6. Security Implementation

- Password encryption using hashing
- Secure authentication system
- Input validation for APIs
- Protection against SQL injection